

Phi-Data Implementation Guide

Table of Contents

1. Introduction
2. Single Prompt vs. Chat Prompt
3. Setting Up Your Environment
4. Implementing Single Prompts with Phi-Data
 - Ollama with Single Prompt
 - Gemini with Single Prompt
 - OpenAI with Single Prompt
5. Implementing Chat Prompts with Phi-Data
 - Ollama with Chat Prompt
 - Gemini with Chat Prompt
 - OpenAI with Chat Prompt
6. Implementing RAG with Phi-Data
 - Chroma RAG with Single Prompt using OpenAI
 - Chroma RAG with Chat Prompt using OpenAI
 - Chroma RAG with Gemini (Single Prompt)
 - Chroma RAG with Gemini (Chat Prompt)
 - Ollama with Single Prompt Example
 - Ollama with Chat Prompt Example

1. Introduction

Phi-Data is a structured methodology for organizing information when working with large language models. It separates information into three main components:

1. **Context:** Background information about the situation or user
2. **Content:** Specific facts or data relevant to the task
3. **Query:** The exact question or request for the model

This structured approach solves common problems with AI interactions:

- Helps models distinguish between background information and actual questions
- Creates clear relationships between different pieces of information
- Improves accuracy by focusing the model on relevant information
- Maintains context in multi-turn conversations
- Works effectively with knowledge retrieval systems

Phi-Data has proven to significantly improve response quality across customer support, content creation, data analysis, and technical documentation.

2. Single Prompt vs. Chat Prompt

When implementing Phi-Data, you'll choose between single prompt and chat prompt approaches based on your specific needs.

Single Prompt

Single prompts are structured one-time interactions where all necessary information is included in a single message. This approach is ideal for:

- **Standalone tasks:** When you need a complete response in one interaction
- **Content creation:** Generating articles, reports, or summaries
- **Data analysis:** Processing information and providing insights
- **Complex instructions:** When providing detailed requirements for a specific output

In single prompts, the Phi-Data structure includes comprehensive context and content sections to provide all necessary information upfront.

Chat Prompt

Chat prompts involve structured ongoing conversations where context builds across multiple exchanges. This approach is better for:

- **Interactive assistance:** Tutoring, customer support, or consultative discussions
- **Progressive refinement:** Iteratively improving outputs based on feedback
- **Complex problem-solving:** Breaking down problems into manageable steps
- **Exploration:** When the end goal might evolve through conversation

In chat prompts, Phi-Data uses structured follow-ups to maintain continuity between messages, preserving context while adding new information or questions.

Key Differences

Aspect	Single Prompt	Chat Prompt
Context Handling	All context provided upfront	Context builds progressively
Information Volume	Higher in initial prompt	Distributed across messages
Typical Structure	Complete structure with all sections	Initial structure followed by structured follow-ups
Interaction Model	One-shot request and response	Multiple turns with feedback loops
Implementation Complexity	Simpler to implement	Requires maintaining conversation state

Choosing the right approach depends on your use case, but both methods benefit from the structured organization that Phi-Data provides.

3. Setting Up Your Environment

Before implementing Phi-Data with different models, you'll need to set up your environment and obtain the necessary API keys.

Installation Requirements

First, install the required packages:

```
# Core packages
pip install phi-agent==2.7.10 chromadb==1.0.13
pip install -U phidata

# For OpenAI
pip install openai==1.88.0

# For Google Gemini
pip install google-generativeai==0.8.5

# For vectorization (if using RAG)
pip install sentence-transformers==4.1.0

# For Ollama (local installation)
# See Ollama setup instructions below
```

API Keys and Setup

OpenAI Setup

1. Create an account at OpenAI's website
2. Generate an API key in your account settings
3. Store your API key in an environment variable:

```
export OPENAI_API_KEY="your-api-key-here"
```

Or in your Python code (not recommended for production):

```
import os
os.environ["OPENAI_API_KEY"] = "your-api-key-here"
```

Google Gemini Setup

1. Create an account on Google AI Studio
2. Generate an API key in the API section
3. Store your API key:

```
export GOOGLE_API_KEY="your-api-key-here"
```

Or in your Python code:

```
import os
os.environ["GOOGLE_API_KEY"] = "your-api-key-here"
```

Ollama Setup

Ollama runs locally on your machine, so you don't need an API key, but you do need to install it:

1. Download and install Ollama from ollama.ai

- Current version: 0.1.29 (as of writing)
- Available for macOS, Linux, and Windows

2. Once installed, pull the Llama model you want to use:

```
# For the 8B parameter model (faster but less capable)
ollama pull llama3:8b

# For the 8B parameter model (better quality)
ollama pull llama3.1

# For best performance (requires more RAM)
ollama pull llama3:70b
```

3. Start the Ollama server:

```
ollama serve
```

4. The server runs on localhost:11434 by default, which the phi-agent library will connect to automatically

4. Implementing Single Prompts with Phi-Data

Ollama with Single Prompt

Implementation:

```
import os
from phi.agent import Agent
from phi.model.ollama import Ollama

# Initialize the agent
# Note: Ollama must be running locally on the default port (11434)
# Available models: llama3.1, llama3:8b, llama3:70b, mistral, phi3, etc.
agent = Agent(model=Ollama(id="llama3.1"), markdown=True)

# Create a structured Phi-Data prompt
phi_data_prompt = """
Context: I'm a biology teacher preparing a lesson on photosynthesis for 10th-grade students.
My students have basic knowledge of cell biology but haven't studied biochemical processes in detail.

Content: Photosynthesis is the process used by plants to convert light energy into chemical energy.
The basic equation is: 6CO2 + 6H2O + light energy → C6H12O6 + 6O2

Query: Create a 5-minute introduction to photosynthesis that will engage teenage students.
Include 2-3 interesting facts that will capture their attention.
"""

# Get response
response = agent.run(phi_data_prompt)
print(response.content)
```

How to Use with Ollama:

1. Install Ollama: Download from ollama.ai and install it on your machine

2. Run Ollama Server:

- Open a terminal and run `ollama serve`
- Leave this terminal window open while you're using Ollama
- You should see a message like "Ollama is running at http://localhost:11434"

3. Download Model:

- In a new terminal, run `ollama pull llama3.1`

- Wait for the model to download (this may take several minutes depending on your internet speed)

4. Run Your Python Script:

- Create a new file called `ollama_phi_data.py` with the code above
- Run it with `python ollama_phi_data.py`
- The script will connect to your local Ollama server, send the prompt, and display the response

5. Troubleshooting:

- If you get a connection error, make sure Ollama is running in another terminal
- If you get a model not found error, make sure you've pulled the model with `ollama pull llama3.1`
- You can check available models with `ollama list`

Explanation: This implementation uses the phi framework to create an agent with Ollama's LLM running locally on your machine. The prompt is structured with clear sections for context, content, and query. The agent processes this structured information and returns a response.

Example Situation: For a teacher preparing a biology lesson, you structure a prompt with:

- Context: Who you are and information about your audience
- Content: Basic facts about photosynthesis
- Query: A specific request for a short, engaging introduction

Gemini with Single Prompt

Implementation:

```
import os

from phi.agent import Agent
from phi.model.google import Gemini

# Set your API key
os.environ["GOOGLE_API_KEY"] = "your-gemini-api-key-here"

# Initialize the agent
# Available models: gemini-1.5-pro, gemini-1.5-flash, gemini-1.0-pro
agent = Agent(model=Gemini(id="gemini-1.5-flash"), markdown=True)

# Create a structured Phi-Data prompt
phi_data_prompt = """
Context:
Company: EcoTech Solutions
Industry: Renewable Energy
Founded: 2018
Size: 120 employees
Target audience: Commercial real estate companies

Content:
Service name: SolarOptimize
Description: AI-powered solar panel installation and optimization service that:
- Analyzes building structure and sun exposure patterns
- Determines optimal panel placement and type
- Predicts energy generation and ROI
- Monitors performance and suggests adjustments

Benefits:
- Average 15% higher energy generation than standard installations
- Typical ROI in 3.5 years instead of industry average 5.2 years
- Automatic adjustment recommendations increase panel lifespan

Query:
Write a concise marketing email (250-300 words) to commercial real estate companies
introducing our SolarOptimize service. Emphasize the ROI benefits and use a professional
but engaging tone. Include a clear call to action for scheduling a consultation.
"""

# Get response
response = agent.run(phi_data_prompt)
print(response.content)
```

Explanation: This implementation creates an agent using Google's Gemini model. The structured approach helps Gemini understand exactly what information is context (company information), what is content (service details), and what is being asked (marketing email request). This is particularly useful for marketing or creative requests where context is important.

Example Situation: For a marketing email, you structure a prompt with:

- Context: Company profile and target audience information
- Content: Service details, features, and benefits
- Query: Specific request for a marketing email with clear parameters

OpenAI with Single Prompt

Implementation:

```
import os
from phi.agent import Agent
from phi.model.openai import OpenAIChat

# Set your API key
os.environ["OPENAI_API_KEY"] = "your-openai-api-key-here"

# Initialize the agent
# Available models: gpt-4o, gpt-4, gpt-3.5-turbo, etc.
agent = Agent(model=OpenAIChat(id="gpt-4o"), markdown=True)

# Create a structured Phi-Data prompt
phi_data_prompt = """
Context:
Audience: Marketing team with limited data science background
Needs: Understanding key customer behavior metrics
Purpose: Quarterly strategy planning

Content:
Customer Acquisition Cost (CAC):
- Current: $42.18
- Previous quarter: $38.75
- Year ago: $35.22

Conversion Rate:
- Current: 3.2%
- Previous quarter: 2.9%
- Year ago: 3.5%

Average Order Value (AOV):
- Current: $78.40
- Previous quarter: $72.15
- Year ago: $68.30

Customer Lifetime Value (CLV):
- Current: $254.80
- Previous quarter: $245.60
- Year ago: $232.10

Query:
Analyze these customer metrics and explain what they mean for our business in simple terms.
Identify concerning trends and suggest 2-3 areas to focus on in the next quarter.
Use analogies or examples to make the concepts accessible to non-technical team members.
"""

# Get response
response = agent.run(phi_data_prompt)
print(response.content)
```

Explanation: This implementation creates an agent using OpenAI's GPT-4o model. The structured approach helps organize complex data analysis requests by clearly separating the audience information, the metrics data, and the specific analysis request. This is especially valuable for data analysis tasks, where properly organizing numerical information helps generate more relevant and accessible insights.

Example Situation: For a data analysis task, you structure a prompt with:

- Context: Information about the audience and purpose
- Content: Structured metrics data with current values and historical comparisons
- Query: Request for analysis with specific requirements

5. Implementing Chat Prompts with Phi-Data

Ollama with Chat Prompt

Implementation:

```
from phi.agent import Agent
from phi.model.ollama import Ollama
from phi.chat import Chat

# Initialize agent and chat
# Note: Ollama must be running locally on the default port (11434)
# Available models: llama3.1, llama3:8b, llama3:70b, mistral, phi3, etc.
agent = Agent(model=Ollama(id="llama3.1"), markdown=True)
chat = Chat(agent)

# Initial structured message
initial_message = """
Context:
Student profile:
- Grade: 9th Grade
- Subject: Algebra I
- Strengths: Good with basic arithmetic and solving simple equations
- Struggles: Difficulty with word problems and systems of equations

Content:
Current problem:
A landscaping company charges $50 per hour plus $200 for materials to install a garden.
A competing company charges $65 per hour plus $125 for materials for the same job.

Query:
How many hours of work would make the total cost of both companies equal?
Help me solve this problem. I'm not sure how to set up the equation.
"""

response1 = chat.send_message(initial_message)
print("Tutor (Response 1):", response1)

# Follow-up structured message
follow_up = """
Follow-up:
Context: I tried setting it up like this:
50h + 200 = 65h + 125
Then I moved the h terms to one side:
50h - 65h = 125 - 200
-15h = -75
Then I divided both sides by -15:
h = -75/-15 = 5

Query:
So it's 5 hours, right? Could you explain what this answer means in the context of the problem?
"""

response2 = chat.send_message(follow_up)
print("\nTutor (Response 2):", response2)
```

Interactive Implementation: For a more interactive experience, you can modify the script to accept user input:

```

from phi.agent import Agent
from phi.model.ollama import Ollama
from phi.chat import Chat

# Initialize agent and chat
agent = Agent(model=Ollama(id="llama3.1"), markdown=True)
chat = Chat(agent)

# Initial structured message
initial_message = """
Context:
Student profile:
- Grade: 9th Grade
- Subject: Algebra I
- Strengths: Good with basic arithmetic and solving simple equations
- Struggles: Difficulty with word problems and systems of equations

Content:
Current problem:
A landscaping company charges $50 per hour plus $200 for materials to install a garden.
A competing company charges $65 per hour plus $125 for materials for the same job.

Query:
How many hours of work would make the total cost of both companies equal?
Help me solve this problem. I'm not sure how to set up the equation.
"""

response = chat.send_message(initial_message)
print("\nTutor:", response)

# Interactive chat loop
while True:
    user_input = input("\nYour message (or 'exit' to quit): ")
    if user_input.lower() == 'exit':
        break

    response = chat.send_message(user_input)
    print("\nTutor:", response)

```

Explanation: This implementation creates a chat session with Ollama. The initial message provides student profile information, the specific problem, and the request for help. The follow-up message shows the student's work and asks for verification and explanation. This structure helps the tutor understand the student's background, track their progress, and provide targeted guidance.

Example Situation: For a math tutoring conversation, you structure messages with:

- Initial message: Student profile, current problem, and specific request
- Follow-up: Student's work and specific questions about their solution

Gemini with Chat Prompt

Implementation:

```

import os

from phi.agent import Agent
from phi.model.google import Gemini
from phi.chat import Chat

# Set your API key
os.environ["GOOGLE_API_KEY"] = "your-gemini-api-key-here"

# Initialize agent and chat
# Available models: gemini-1.5-pro, gemini-1.5-flash, gemini-1.0-pro
agent = Agent(model=Gemini(id="gemini-1.5-flash"), markdown=True)
chat = Chat(agent)

# Initial structured message
initial_message = """
Context:
Traveler profile:
- Group: Family of 4 (parents with children ages 8 and 11)
- Travel style: Active but not too strenuous, mix of culture and nature
- Interests: History, outdoor activities, local cuisine, family-friendly attractions
- Budget: Moderate ($3000-$4000 for 7 days excluding flights)
- Constraints: No extreme heat, nothing too physically demanding for children

Content:
Travel request:
- Destination: Japan
- Duration: 7 days
- Season: Spring (April)

Query:
We're planning our first trip to Japan. Which cities/regions would you recommend
for a 7-day itinerary that's good for families? We want to see cherry blossoms but avoid
the most crowded tourist areas if possible.
"""

response1 = chat.send_message(initial_message)
print("Travel Advisor (Response 1):", response1)

# Follow-up structured message
follow_up = """
Follow-up:
Context: Thanks for those suggestions! Tokyo and Kyoto look perfect for our trip.
We'd like to focus on these two cities since we don't want to rush too much with the kids.

Query:
Additional questions:
1. What are some specific kid-friendly activities in Tokyo and Kyoto?
2. How should we get around? Is a rail pass worth it if we're just doing Tokyo and Kyoto?
3. Any recommendations for where to see cherry blossoms that might be less crowded?
"""

response2 = chat.send_message(follow_up)
print("\nTravel Advisor (Response 2):", response2)

```

Explanation: This implementation creates a chat session with Google's Gemini model. The initial message provides detailed traveler profile information and a specific travel request. The follow-up message includes feedback on the initial suggestions and specific follow-up questions about the refined plans. This structure helps maintain the conversation flow while ensuring all relevant context is preserved.

Example Situation: For a travel planning conversation, you structure messages with:

- Initial message: Traveler profile and specific travel request
- Follow-up: Feedback on initial recommendations and new specific questions

OpenAI with Chat Prompt

Implementation:

```

import os

from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.chat import Chat

# Set your API key
os.environ["OPENAI_API_KEY"] = "your-openai-api-key-here"

# Initialize agent and chat
# Available models: gpt-4o, gpt-4, gpt-3.5-turbo, etc.
agent = Agent(model=OpenAIChat(id="gpt-4o"), markdown=True)
chat = Chat(agent)

# Initial structured message
initial_message = """
Context:
Company context:
- Company name: HealthTrack
- Industry: Health and fitness technology
- Product: Mobile app for fitness tracking and nutrition planning
- Target audience: Adults 25-45 interested in improving fitness and nutrition
- Current metrics: 250,000 monthly active users, 65% retention after 3 months, 3.7/5 app store rating

Content:
Product issues:
1. High priority: User engagement drops significantly after 4-6 weeks of use
   User feedback: "It gets repetitive and doesn't keep me motivated long-term"
2. Medium priority: Nutrition tracking feature has high abandonment rate
   User feedback: "Too time-consuming to log meals accurately"
3. Medium priority: Users struggle to understand progress metrics
   User feedback: "I can see the data but don't know what it means for my goals"

Query:
What product improvements would you recommend to address our user engagement and retention issues?
We're particularly interested in solutions that could be implemented within a 3-month development cycle.
"""

response1 = chat.send_message(initial_message)
print("Consultant (Response 1):", response1)

# Follow-up structured message
follow_up = """
Follow-up:
Context: Thanks for these suggestions! The personalized milestone system and simplified nutrition tracking
seem especially promising. Our development team has some questions about implementation.

Content:
Development constraints:
- Our AI capabilities are limited, but we have a data science team
- We can only integrate with 2-3 third-party services in this cycle
- Any new features must work on both iOS and Android platforms

Query:
Specific questions:
1. For the personalized milestone system, what data points should we prioritize tracking to make it effective without overwhelming users?
2. Which third-party nutrition databases or APIs would you recommend integrating with for the simplified nutrition tracking?
3. How should we measure success for these new features? What metrics would indicate they're working as intended?
"""

response2 = chat.send_message(follow_up)
print("\nConsultant (Response 2):", response2)

```

Explanation: This implementation creates a chat session with OpenAI's GPT-4o model. The structured approach helps with complex consultative conversations where context is continually building. The initial message provides company context and specific product issues, while the follow-up adds new constraints and asks specific implementation questions. This helps ensure the model has comprehensive context for providing practical, actionable advice.

Example Situation: For a product development conversation, you structure messages with:

- Initial message: Company context, product issues, and general request for recommendations
- Follow-up: Feedback on initial suggestions, new constraints, and specific implementation questions

6. Implementing RAG with Phi-Data

Chroma RAG with Single Prompt using OpenAI

Implementation:

```

import os
import chromadb
from phi.agent import Agent
from phi.model.openai import OpenAIChat

# Set your OpenAI API key
os.environ["OPENAI_API_KEY"] = "your-openai-api-key-here"

# Setup Chroma client (version 0.4.22)
client = chromadb.Client()

# Sample documents with structured data
structured_document1 = """
Document ID: POL001
Title: Remote Work Policy
Content:
TechCorp allows employees to work remotely up to 3 days per week.
Employees must be available during core hours (10am-3pm).
Department managers may approve full-time remote work on a case-by-case basis.
"""

structured_document2 = """
Document ID: POL002
Title: Business Travel Policy
Content:
All business travel must be approved in advance by the department head.
Economy class should be booked for all flights under 5 hours.
Expense reports must be submitted within 14 days of return.
"""

# Create a collection and add documents
collection = client.create_collection(name="knowledge_base")
collection.add(documents=[structured_document1, structured_document2], ids=["doc1", "doc2"])

# Initialize agent
agent = Agent(model=OpenAIChat(id="gpt-4o"), markdown=True)

# Function to query with Phi-Data structure
def query_knowledge_base(question, user_context="Employee seeking policy information"):
    # Retrieve relevant documents
    results = collection.query(query_texts=[question], n_results=2)

    # Format retrieved documents into Phi-Data structure
    phi_data_prompt = f"""
    Context: {user_context}

    Content:
    Retrieved information:
    {results["documents"][0]}

    Query: {question}

    Requirements:
    - Use only information from the retrieved content
    - Be clear and concise
    - Cite the source of information
    """

    # Generate response
    response = agent.run(phi_data_prompt)
    return response.content

# Example usage
response = query_knowledge_base(
    question="How many days per week can I work from home?",

```

```
        user_context="New employee orientation"  
    )  
    print(response)
```

Explanation: This implementation creates a RAG system using Chroma as the vector database. Documents stored with structured format are retrieved based on a query, then formatted into a Phi-Data structure with both the original query and the retrieved information. This helps the model generate responses grounded in the retrieved context.

Chroma RAG with Chat Prompt using OpenAI

Implementation:

```

import os
import chromadb
from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.chat import Chat

# Set your OpenAI API key
os.environ["OPENAI_API_KEY"] = "your-openai-api-key-here"

# Setup Chroma client
client = chromadb.Client()

# Sample documents with structured data
structured_document1 = """
Document ID: POL001
Title: Remote Work Policy
Content:
TechCorp allows employees to work remotely up to 3 days per week.
Employees must be available during core hours (10am-3pm).
Department managers may approve full-time remote work on a case-by-case basis.
"""

structured_document2 = """
Document ID: POL002
Title: Remote Work Request Process
Content:
To request full-time remote work:
1. Complete the Remote Work Request Form
2. Provide justification for the request
3. Obtain your direct manager's approval
4. Submit to HR for final review
Decisions are typically made within 5 business days.
"""

# Create a collection and add documents
collection = client.create_collection(name="knowledge_base")
collection.add(documents=[structured_document1, structured_document2], ids=["doc1", "doc2"])

# Initialize agent and chat
agent = Agent(model=OpenAIChat(id="gpt-4o"), markdown=True)
chat = Chat(agent)

# Function to retrieve documents
def retrieve_docs(query):
    results = collection.query(query_texts=[query], n_results=2)
    return results["documents"][0]

# Initial question
initial_question = "What is the company's remote work policy?"

# Retrieve relevant documents
docs = retrieve_docs(initial_question)

# Create Phi-Data structured prompt with retrieved information
prompt_with_docs = f"""
Context: New employee orientation

Content:
Retrieved information:
{docs}

Query: {initial_question}
"""

# Get initial response

```

```

response1 = chat.send_message(prompt_with_docs)
print("HR Assistant (Response 1):", response1)

# Follow-up question
follow_up_question = "How do I request full-time remote work?"

# Retrieve new relevant documents
new_docs = retrieve_docs(follow_up_question)

# Create Phi-Data structured follow-up prompt
follow_up_with_docs = f"""
Follow-up:
Context: I have childcare responsibilities two days a week

Content:
Retrieved information:
{new_docs}

Query: {follow_up_question}
"""

# Get follow-up response
response2 = chat.send_message(follow_up_with_docs)
print("\nHR Assistant (Response 2):", response2)

```

Explanation: This implementation creates a chat-based RAG system that retrieves relevant documents for each question in the conversation. For each new message, it retrieves fresh information from the knowledge base and incorporates it into a Phi-Data structured prompt. This allows for contextual, knowledge-grounded conversations that build on previous exchanges.

Chroma RAG with Gemini (Single Prompt)

Implementation:

```

import os
import chromadb
from phi.agent import Agent
from phi.model.google import Gemini

# Set your Google API key
os.environ["GOOGLE_API_KEY"] = "your-gemini-api-key-here"

# Setup Chroma client (version 0.4.22)
client = chromadb.Client()

# Sample documents
doc1 = """
Title: Python Basics Guide
Python is a high-level, interpreted programming language known for its readability and simplicity.
Key features:
- Easy to learn syntax
- Dynamically typed
- Garbage collected
- Supports multiple programming paradigms
- Extensive standard library

Popular libraries include NumPy for numerical computing, Pandas for data analysis,
and TensorFlow/PyTorch for machine learning.

Basic syntax example:
def greet(name):
    return f"Hello, {name}!"
message = greet("World")
print(message) # Outputs: Hello, World!
"""

doc2 = """
Title: Python Best Practices
Follow these guidelines when writing Python code:
1. Follow PEP 8 style guide for consistent code formatting
2. Use virtual environments to manage dependencies
3. Write descriptive docstrings for functions and classes
4. Use list comprehensions for creating lists
5. Utilize context managers (with statement) for file operations
6. Catch specific exceptions rather than generic ones
7. Use f-strings for string formatting (Python 3.6+)
8. Leverage built-in functions like map, filter, zip where appropriate

Example of good practices:
def calculate_average(numbers):
    """
    Calculate the average of a list of numbers.
    Args: numbers (list): A list of numbers
    Returns: float: The average value
    """
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)
"""

# Create a collection and add documents
collection = client.create_collection(name="python_docs")
collection.add(documents=[doc1, doc2], ids=["basics", "best_practices"])

# Initialize agent
agent = Agent(model=Gemini(id="gemini-1.5-flash"), markdown=True)

def query_python_docs(question, user_context="Python developer seeking guidance"):
    # Retrieve relevant documents
    results = collection.query(query_texts=[question], n_results=1)

```

```

# Format Phi-Data prompt
prompt = f"""
Context: {user_context}

Content:
Retrieved documentation:
{results["documents"][0]}

Query: {question}

Instructions:
- Answer based only on the information in the documentation above
- If the information doesn't contain the answer, say so
- Use code examples when appropriate
- Be clear and concise
"""

# Generate response
response = agent.run(prompt)
return response.content

# Example usage
response = query_python_docs(
    question="What are the key features of Python?",
    user_context="Beginner programmer learning Python"
)
print(response)

```

Chroma RAG with Gemini (Chat Prompt)

Implementation:

```

import os
import chromadb
from phi.agent import Agent
from phi.model.google import Gemini
from phi.chat import Chat

# Set your Google API key
os.environ["GOOGLE_API_KEY"] = "your-gemini-api-key-here"

# Setup Chroma client
client = chromadb.Client()

# Sample documents
doc1 = """
Title: Python Basics Guide
Python is a high-level, interpreted programming language known for its readability and simplicity.
Key features:
- Easy to learn syntax
- Dynamically typed
- Garbage collected
- Supports multiple programming paradigms
- Extensive standard library

Popular libraries include NumPy for numerical computing, Pandas for data analysis,
and TensorFlow/PyTorch for machine learning.
"""

doc2 = """
Title: Python Best Practices
Follow these guidelines when writing Python code:
1. Follow PEP 8 style guide for consistent code formatting
2. Use virtual environments to manage dependencies
3. Write descriptive docstrings for functions and classes
4. Use list comprehensions for creating lists
5. Utilize context managers (with statement) for file operations
6. Catch specific exceptions rather than generic ones
7. Use f-strings for string formatting (Python 3.6+)
8. Leverage built-in functions like map, filter, zip where appropriate
"""

# Create a collection and add documents
collection = client.create_collection(name="python_docs")
collection.add(documents=[doc1, doc2], ids=["basics", "best_practices"])

# Initialize agent and chat
agent = Agent(model=Gemini(id="gemini-1.5-flash"), markdown=True)
chat = Chat(agent)

def retrieve_docs(question):
    results = collection.query(query_texts=[question], n_results=1)
    return results["documents"][0]

# Initial query
question = "What are the key features of Python?"
docs = retrieve_docs(question)

# Format initial Phi-Data prompt with retrieved information
initial_prompt = f"""
Context: I'm a Python developer with a question.

Content:
Relevant documentation:
{docs}

Query: {question}

```

```

Please answer based on this documentation.
"""

# Send initial prompt and get response
response1 = chat.send_message(initial_prompt)
print("Response 1:", response1)

# Follow-up question
follow_up = "What libraries are commonly used in Python?"

# Get new relevant documentation for follow-up
follow_up_docs = retrieve_docs(follow_up)

# Create Phi-Data structured follow-up prompt
follow_up_prompt = f"""
Follow-up:
Context: I want to understand Python's ecosystem better.

Content:
Relevant documentation:
{follow_up_docs}

Query: {follow_up}
Please answer based on this documentation.
"""

# Send follow-up and get response
response2 = chat.send_message(follow_up_prompt)
print("\nResponse 2:", response2)

```

Ollama with Single Prompt Example

Implementation:

```

import chromadb

from phi.agent import Agent
from phi.model.ollama import Ollama

# Setup Chroma client (version 0.4.22)
client = chromadb.Client()

# Sample documents with structured data
structured_document1 = """
Document ID: TECH001
Title: Machine Learning Basics
Content:
Machine learning is a subset of artificial intelligence that enables systems to learn from data
rather than through explicit programming. The three main types are supervised learning,
unsupervised learning, and reinforcement learning. Supervised learning uses labeled training
data, while unsupervised learning finds patterns in unlabeled data.
"""

structured_document2 = """
Document ID: TECH002
Title: Neural Networks Overview
Content:
Neural networks are computing systems inspired by biological neural networks in animal brains.
They consist of layers of interconnected nodes or "neurons" that process information.
Deep learning refers to neural networks with multiple hidden layers. Common architectures
include CNNs for image processing and RNNs for sequential data like text.
"""

# Create a collection and add documents
collection = client.create_collection(name="tech_knowledge_base")
collection.add(documents=[structured_document1, structured_document2], ids=["doc1", "doc2"])

# Initialize agent
# Note: Ollama must be running locally on the default port (11434)
agent = Agent(model=Ollama(id="llama3.1"), markdown=True)

# Function to query with Phi-Data structure
def query_knowledge_base(question, user_context="Student learning about artificial intelligence concepts"):
    # Retrieve relevant documents
    results = collection.query(query_texts=[question], n_results=1)

    # Format retrieved documents into Phi-Data structure
    structured_prompt = f"""
Context: {user_context}

Content:
{results["documents"][0]}

Query: {question}
Please explain using simple language and provide 1-2 examples to illustrate the concept.
"""

    # Generate response
    response = agent.run(structured_prompt)
    return response.content

# Example usage
response = query_knowledge_base(
    question="What is machine learning and what are its main types?",
    user_context="First-year computer science student new to AI concepts"
)
print(response)

```

Explanation: This implementation creates a RAG system using Chroma as the vector database and Ollama as the local LLM. The structured prompt follows the Phi-Data format, clearly delineating context, content (the retrieved document), and the specific query. The system retrieves relevant documents based on the user's question and formats them into a Phi-Data structured prompt before passing to the Ollama model.

Ollama with Chat Prompt Example

Implementation:

```

import chromadb
from phi.agent import Agent
from phi.model.ollama import Ollama
from phi.chat import Chat

# Setup Chroma client
client = chromadb.Client()

# Sample documents
structured_document1 = """
Document ID: TECH001
Title: Machine Learning Basics
Content:
Machine learning is a subset of artificial intelligence that enables systems to learn from data
rather than through explicit programming. The three main types are supervised learning,
unsupervised learning, and reinforcement learning. Supervised learning uses labeled training
data, while unsupervised learning finds patterns in unlabeled data.
"""

structured_document2 = """
Document ID: TECH002
Title: Neural Networks Overview
Content:
Neural networks are computing systems inspired by biological neural networks in animal brains.
They consist of layers of interconnected nodes or "neurons" that process information.
Deep learning refers to neural networks with multiple hidden layers. Common architectures
include CNNs for image processing and RNNs for sequential data like text.
"""

structured_document3 = """
Document ID: TECH003
Title: Applications of Machine Learning
Content:
Machine learning powers many modern applications including:
1. Image and speech recognition systems
2. Recommendation algorithms on streaming and shopping platforms
3. Natural language processing for chatbots and translation
4. Fraud detection in financial transactions
5. Autonomous vehicles and predictive maintenance in manufacturing
"""

# Create a collection and add documents
collection = client.create_collection(name="tech_knowledge_base")
collection.add(documents=[structured_document1, structured_document2, structured_document3],
               ids=["doc1", "doc2", "doc3"])

# Initialize agent and chat
# Note: Ollama must be running locally on the default port (11434)
agent = Agent(model=Ollama(id="llama3.1"), markdown=True)
chat = Chat(agent)

# Function to retrieve documents
def retrieve_docs(query):
    results = collection.query(query_texts=[query], n_results=1)
    return results["documents"][0]

# Initial question
initial_question = "What is machine learning?"

# Retrieve relevant documents
docs = retrieve_docs(initial_question)

# Create first Phi-Data structured prompt with retrieved information
initial_prompt = f"""
Context:

```

```

Student profile:
- Educational level: First-year computer science student
- Background: Basic programming knowledge but new to AI concepts
- Learning style: Prefers explanations with real-world examples

Content:
Retrieved information:
{docs}

Query: {initial_question}

Instructions:
- Explain the concept in simple terms
- Provide at least one real-world example
- Identify any terms that might need further explanation
"""

# Get initial response
response1 = chat.send_message(initial_prompt)
print("Tutor (Response 1):", response1)

# Follow-up question
follow_up_question = "What are some real-world applications of machine learning?"

# Retrieve new relevant documents
new_docs = retrieve_docs(follow_up_question)

# Create Phi-Data structured follow-up prompt
follow_up_prompt = f"""
Follow-up:
Context: Building on our previous discussion about machine learning basics.

Content:
Retrieved information:
{new_docs}

Query: {follow_up_question}

Instructions:
- Connect this answer to our previous discussion about machine learning basics
- Explain how the different types of machine learning (supervised, unsupervised) apply to these applications
- Keep the explanation at an introductory level
"""

# Get follow-up response
response2 = chat.send_message(follow_up_prompt)
print("\nTutor (Response 2):", response2)

```

Explanation: This implementation creates a chat-based RAG system using Ollama as the LLM and Chroma as the vector database. The structured approach helps with educational conversations by clearly separating the student profile, the retrieved information, and specific instructions for the tutor. For each new question in the conversation, it retrieves fresh information from the knowledge base and incorporates it into a new Phi-Data structured prompt, while maintaining the context of the ongoing conversation.

Example Situation: For an AI tutoring conversation, you structure messages with:

- Initial message: Student profile, initial question, and retrieved reference material
- Follow-up: New question, freshly retrieved information, and instructions that reference previous context

Best Practices and Tips

1. Context Design

- **Be specific** about the user's background, experience level, and needs
- **Include relevant constraints** that affect the response (time, budget, technical limitations)
- **Specify the audience** if the output is intended for someone else

2. Content Organization

- **Structure information clearly** with headers, bullet points, or numbered lists
- **Include all relevant data** but avoid information overload
- **Cite sources** when using retrieved information in RAG implementations

3. Query Formulation

- **Be specific** about desired output format, length, and style
- **Include clear instructions** about what to emphasize or avoid
- **Specify how to handle** missing information or edge cases

4. RAG Integration

- **Choose appropriate chunk sizes** for your documents
- **Use meaningful metadata** to improve retrieval accuracy
- **Test different embedding models** for your specific domain

5. Error Handling

```
def safe_query(agent, prompt):  
    try:  
        response = agent.run(prompt)  
        return response.content  
    except Exception as e:  
        return f"Error processing request: {str(e)}"
```

6. Performance Optimization

- **Cache frequently used responses** to reduce API calls
- **Use appropriate model sizes** for your use case (balance cost vs. quality)
- **Implement rate limiting** for production applications

This comprehensive guide provides a structured approach to implementing Phi-Data across different LLM providers and use cases, ensuring consistent, high-quality AI interactions.